

Peridynamics With Softening: Model and Code Details

Contents

1	Purpose of This Document	3
2	Source File Map	3
3	Build and Run	3
4	Geometry and Discretization	4
5	Horizon and Peridynamic Families	5
6	Material Parameters	5
7	Bond Micromodulus	6
8	Surface and Volume Corrections	6
9	Kinematics	6
10	Load Control and Boundary Conditions	7
10.1	Load Predictor	7
10.2	Displacement-Prescribed Grip Nodes	8
10.3	Body Forces	8
11	Virtual Substrate Coupling	8
12	Bar-Bar Bond Force	9
13	Damage and Softening Models	10
13.1	Code-to-Equation Mapping in the Bond Loop	10
13.2	Common Elastic Contribution	11
13.3	Common Softening Contribution	11
13.4	Common Complete-Failure Contribution	12
13.5	Critical Stretch	12
13.6	Elastic Model	13
13.7	Brittle Model	13
13.8	Linear Softening Model	14
13.9	Bilinear Softening Model	14
13.10	Cornelissen Softening Model	14
13.11	Exponential Softening Model	15

14 Nodal Damage Variable	15
15 Stress Measure	15
16 Output Strain	15
17 Energy Quantities	16
18 Dynamic Relaxation Solver	17
19 Damping and Convergence	17
20 Algorithm Summary	18
21 Output Files	19
21.1 Configuration Files	19
21.2 OVITO Files	19
21.3 Final ADR Field Matrices	19
21.4 Energy Histories	19
21.5 Stress Convergence	20
21.6 Snapshots and Diagnostics	20
22 First-Crack Early Stop	20
23 Defect Handling	20
24 Post-Processing	21
25 Active Versus Dormant Code	21
26 Important Implementation Notes	21

1 Purpose of This Document

This document explains the active C++ model in the repository: the implemented equations, geometry, boundary conditions, softening and failure laws, numerical relaxation scheme, output files, and the role of each source file.

The current executable is a one-dimensional tensile peridynamic bar. By default the left grip is fixed, the right grip is pulled in displacement control, and the virtual substrate coupling is disabled. Several older two-dimensional, thermal, and explicit film-substrate-grid ideas remain in the source as comments, but the active calculation uses material points on a line,

$$y_i = 0, \quad i = 0, \dots, N - 1. \quad (1)$$

2 Source File Map

File	Purpose
<code>main.cpp</code>	Defines simulation parameters, allocates arrays, builds geometry and correction factors, calls the mechanical solver, and frees memory.
<code>Geometry.h</code>	Builds the one-dimensional nodal coordinates and the peridynamic neighbor families.
<code>surface_correction_factors.h</code>	Computes the end-effect correction factors used in the bond integration weights.
<code>matrices.h</code>	Allocates, initializes, precomputes, and frees vector and pointer-based fields.
<code>only_mechanical.h</code>	Contains the main quasi-static mechanical solve: load increments, boundary conditions, bond forces, softening, damage, virtual substrate coupling, dynamic relaxation, energy calculation, and output writing.
<code>DefectUtils.h</code>	Builds a node mask for an optional defect region. In the current force loop, the local defect critical stretch is not applied because that code is commented out.
<code>DiagnosticsUtils.h</code>	Provides helpers for diagnostic-node selection and optional diagnostic traces.
<code>time_settings.h</code>	Defines mechanical and thermal time-step constants. Only the mechanical values are active.
<code>variable_initialization.h</code>	Initializes scalar scratch variables used by the included-style C++ code.
<code>write_ovito_files.h</code>	Legacy OVITO writer. It is not included by the active <code>main.cpp</code> .
<code>plot_strain.py</code>	Post-processing script for plotting or animating <code>result/conf*.dat</code> .
<code>Makefile</code>	Builds the executable <code>peridynamics_with_softening</code> .

3 Build and Run

The executable is built with

make

The default run is

```
./peridynamics_with_softening
```

The executable also accepts optional command-line arguments:

```
./peridynamics_with_softening [increments_to_run] [load_increments] [damage_model]
```

The first argument controls how many outer increments are executed. The second argument controls the denominator used in the strain increment,

$$\Delta\varepsilon = \frac{\varepsilon_{\text{target}}}{N_{\text{load}}}. \quad (2)$$

The third argument selects the damage law:

damage_model	Law
-1	Elastic, no damage
0	Brittle failure
1	Linear softening, default
2	Bilinear softening
3	Cornelissen softening
4	Exponential softening

For example,

```
./peridynamics_with_softening 40 80 3
```

runs 40 increments with the load-step size of an 80-increment ramp using the Cornelissen softening law.

4 Geometry and Discretization

The active domain is a one-dimensional bar of length

$$L = 1. \quad (3)$$

The current number of material points is

$$N = 300. \quad (4)$$

The grid spacing is

$$\Delta x = \frac{L}{N-1}. \quad (5)$$

The material-point coordinates are

$$x_i = i\Delta x, \quad y_i = 0, \quad i = 0, \dots, N-1. \quad (6)$$

The active cross-sectional area is

$$A = 1, \quad (7)$$

so the nodal volume is

$$V = A\Delta x = \Delta x. \quad (8)$$

The C++ variables are:

Quantity	Variable	Current value
Length	<code>length</code>	1
Number of points	<code>ndivx</code> , <code>totnode</code> , <code>totint</code>	300
Area	<code>area_bar</code>	1
Thickness	<code>thickness</code>	1, not active in the force law
Maximum family storage	<code>maxfam</code>	20

5 Horizon and Peridynamic Families

The peridynamic horizon is

$$\delta = 3\Delta x. \quad (9)$$

The horizon ratio is therefore

$$m = \frac{\delta}{\Delta x} = 3. \quad (10)$$

For each material point i , its family is

$$\mathcal{H}_i = \{j \neq i : |x_j - x_i| \leq \delta + 10^{-12}\}. \quad (11)$$

The small 10^{-12} tolerance prevents floating-point roundoff from dropping bonds located exactly at the horizon.

The family data are stored using older one-based indexing conventions:

- `numfam[i]` stores the number of neighbors of node i ;
- `pointfam[i]` stores the one-based offset into `nodefam`;
- `nodefam` stores one-based neighbor node indices.

When the code retrieves a neighbor, it converts the stored index back to a zero-based C++ index.

6 Material Parameters

The current active material values are:

Quantity	Variable	Value
Poisson ratio	<code>pratio</code>	1/3
Film density	<code>rho_film</code>	1
Substrate density	<code>rho_substrate</code>	1, mostly inactive
Film Young's modulus	<code>emod_film</code>	1
Substrate Young's modulus	<code>emod_substrate</code>	1, mostly inactive

The code computes the film plane-stress modulus

$$\bar{E}_f = \frac{E_f}{1 - \nu^2}, \quad (12)$$

and the film shear modulus

$$\mu_f = \frac{E_f}{2(1 + \nu)}. \quad (13)$$

The active one-dimensional bond force law uses the peridynamic bond stiffness below, not the shear modulus directly.

7 Bond Micromodulus

The one-dimensional bond stiffness before discrete correction is

$$c_0 = \frac{2E}{A\delta^2}. \quad (14)$$

The code applies the correction

$$C_d = \frac{m}{m+1}. \quad (15)$$

Thus the active stiffness is

$$c = \frac{2E}{A\delta^2} \frac{m}{m+1}. \quad (16)$$

In the code this is stored as `bc_mechanical_film_bar`.

8 Surface and Volume Corrections

Boundary-near nodes have incomplete horizons. The active geometric correction is based on the visible left and right horizon lengths:

$$\delta_i^- = \min(\delta, x_i), \quad (17)$$

$$\delta_i^+ = \min(\delta, L - x_i). \quad (18)$$

The nodal geometric correction is

$$G_i = \frac{2\delta}{\delta_i^- + \delta_i^+}. \quad (19)$$

The first and last three displacement-prescribed grip nodes are explicitly set to

$$G_i = 1. \quad (20)$$

The bond correction is the average

$$G_{ij} = \frac{G_i + G_j}{2}. \quad (21)$$

The separate volume-correction factor is currently

$$\text{fac}_{ij} = 1. \quad (22)$$

The effective integration weight for a bar-bar bond is

$$w_{ij} = V G_{ij} \text{fac}_{ij}. \quad (23)$$

9 Kinematics

The displacement of material point i is

$$u_i. \quad (24)$$

The current position is

$$y_i = x_i + u_i. \quad (25)$$

For bond (i, j) , the reference separation is

$$\xi_{ij} = x_j - x_i, \quad (26)$$

with length

$$r_{ij}^0 = |\xi_{ij}|. \quad (27)$$

The current separation is

$$\eta_{ij} = (x_j + u_j) - (x_i + u_i), \quad (28)$$

with length

$$r_{ij} = |\eta_{ij}|. \quad (29)$$

The bond stretch is

$$s_{ij} = \frac{r_{ij} - r_{ij}^0}{r_{ij}^0}. \quad (30)$$

The one-dimensional direction cosine is

$$n_{ij} = \frac{\eta_{ij}}{r_{ij}}. \quad (31)$$

The active implementation computes r_{ij} using the x -component only, which is consistent with the active one-dimensional geometry.

For history-dependent damage, the code stores

$$s_{ij}^{\max}(t) = \max_{\tau \leq t} s_{ij}(\tau). \quad (32)$$

This field is `bond_s_history[i][j]`.

10 Load Control and Boundary Conditions

The imposed macroscopic strain starts from

$$\varepsilon_0 = 0. \quad (33)$$

The default target strain is

$$\varepsilon_{\text{target}} = 0.004. \quad (34)$$

The default number of load increments is

$$N_{\text{load}} = 80. \quad (35)$$

The strain increment is

$$\Delta\varepsilon = \frac{\varepsilon_{\text{target}}}{N_{\text{load}}} = 5 \times 10^{-5} \quad \text{for the defaults.} \quad (36)$$

At outer load step k ,

$$\varepsilon_k = \varepsilon_{k-1} + \Delta\varepsilon. \quad (37)$$

10.1 Load Predictor

Before dynamic relaxation begins, the code keeps the previous converged displacement field and adds an affine predictor:

$$u_i \leftarrow u_i + 0.75 \Delta\varepsilon x_i. \quad (38)$$

10.2 Displacement-Prescribed Grip Nodes

The first and last three nodes are prescribed:

$$i = 0, 1, 2, \quad i = N - 3, N - 2, N - 1. \quad (39)$$

During every inner relaxation step, these nodes are reset to

$$u_i = 0, \quad i = 0, 1, 2, \quad (40)$$

and

$$u_i = \varepsilon_k L, \quad i = N - 3, N - 2, N - 1. \quad (41)$$

The prescribed-node velocities are reset to

$$v_i = 0. \quad (42)$$

Therefore the left boundary satisfies

$$u(0) = 0, \quad (43)$$

and the right boundary satisfies

$$u(L) = \varepsilon_k L. \quad (44)$$

The active force loop skips the prescribed nodes, setting their damage, strain energy, substrate energy, and stress to zero.

10.3 Body Forces

The body-force array is initialized to zero and no active body force is applied:

$$b_i = 0. \quad (45)$$

The old traction-style loading code is present only as comments.

11 Virtual Substrate Coupling

The virtual substrate code path is now optional and disabled by default:

$$\text{enable_substrate_coupling} = \text{false}, \quad \text{c_int_ratio} = 0. \quad (46)$$

Therefore, in the simpler tensile-bar scenario,

$$p_i^{\text{sub}} = 0, \quad W_i^{\text{sub}} = 0. \quad (47)$$

If the flag is re-enabled in the source, the substrate is represented by a virtual affine layer, not a separate deformable mesh. It is located a vertical distance

$$h = \Delta x \quad (48)$$

below the bar.

The virtual substrate displacement is prescribed as

$$u_j^{\text{sub}} = \varepsilon_k x_j. \quad (49)$$

For bar node i and virtual substrate point j , the initial interface distance is

$$r_{ij}^{0,\text{sub}} = \sqrt{(x_j - x_i)^2 + h^2}. \quad (50)$$

The interaction is included only when

$$r_{ij}^{0,\text{sub}} \leq \delta + 10^{-12}. \quad (51)$$

The current horizontal separation is

$$d_{ij}^{\text{sub}} = (x_j + u_j^{\text{sub}}) - (x_i + u_i). \quad (52)$$

The current interface distance is

$$r_{ij}^{\text{sub}} = \sqrt{\left(d_{ij}^{\text{sub}}\right)^2 + h^2}. \quad (53)$$

The interface stretch is

$$s_{ij}^{\text{sub}} = \frac{r_{ij}^{\text{sub}} - r_{ij}^{0,\text{sub}}}{r_{ij}^{0,\text{sub}}}. \quad (54)$$

The horizontal direction cosine is

$$n_{ij}^{\text{sub}} = \frac{d_{ij}^{\text{sub}}}{r_{ij}^{\text{sub}}}. \quad (55)$$

The interface stiffness is

$$c_{\text{int}} = \alpha_{\text{int}} c, \quad \alpha_{\text{int}} = 1. \quad (56)$$

When enabled, the virtual substrate force contribution is

$$p_i^{\text{sub}} = \sum_j c_{\text{int}} s_{ij}^{\text{sub}} n_{ij}^{\text{sub}} V. \quad (57)$$

The optional substrate sum uses V directly. It does not use the bar-bar surface correction G_{ij} .

12 Bar-Bar Bond Force

For an intact elastic bond, the force contribution from j to i is

$$f_{ij} = c s_{ij} n_{ij} w_{ij}. \quad (58)$$

The bar-bar internal force is

$$p_i^{\text{bar}} = \sum_{j \in \mathcal{H}_i} f_{ij}. \quad (59)$$

The total force used by dynamic relaxation is

$$p_i = p_i^{\text{bar}} + p_i^{\text{sub}}. \quad (60)$$

13 Damage and Softening Models

The active default is

$$\text{degradation_model} = 1, \quad (61)$$

which means linear softening.

The implemented options are:

Value	Meaning
-1	Elastic, no damage
0	Brittle failure
1	Linear softening
2	Bilinear softening
3	Cornelissen softening
4	Exponential softening
other	Fallback to brittle-style behavior

13.1 Code-to-Equation Mapping in the Bond Loop

For each directed bond from node i to neighbor j , the active code first computes the kinematic quantities

$$r_{ij}^0 = \text{idist}, \quad (62)$$

$$w_{ij} = \mathbf{w_const} = V G_{ij} \text{fac}_{ij}, \quad (63)$$

$$n_{ij} = \mathbf{dirx} = \frac{\eta_{ij}}{|\eta_{ij}|}, \quad (64)$$

$$s_{ij} = \mathbf{stretch} = \frac{|\eta_{ij}| - r_{ij}^0}{r_{ij}^0}. \quad (65)$$

The historical maximum stretch stored in `bond_s_history[i][j]` is updated before the softening branch is evaluated:

$$\hat{s}_{ij}^{n+1} = \max(\hat{s}_{ij}^n, s_{ij}^{n+1}). \quad (66)$$

Below, $\hat{s}_{ij}$ denotes this updated history value.

The code also defines a loading-envelope flag:

$$\text{onEnvelope}_{ij} = (|s_{ij} - \hat{s}_{ij}| < 10^{-14}). \quad (67)$$

When this is true, the current stretch is equal to the historical maximum to roundoff tolerance, so the bond is on the active softening envelope. When it is false, the bond is unloading or reloading below a previously reached maximum.

The force contribution added to node i is the scalar

$$\Delta p_{ij} = \text{dforce1_mechanical}. \quad (68)$$

The bar-bar force accumulator uses

$$p_i^{\text{bar}} \leftarrow p_i^{\text{bar}} + \Delta p_{ij}. \quad (69)$$

The stress moment accumulator uses the same force contribution:

$$M_i \leftarrow M_i + \Delta p_{ij} \xi_{ij}, \quad (70)$$

and the reported stress is

$$\sigma_i = \frac{1}{2} M_i. \quad (71)$$

For nodal damage, the code accumulates two local sums:

$$A_i \leftarrow A_i + q_{ij} V \text{ fac}_{ij}, \quad (72)$$

$$B_i \leftarrow B_i + V \text{ fac}_{ij}, \quad (73)$$

where $q_{ij} = 1$ for intact elastic bonds, $q_{ij} = \mu_{ij}$ for softened bonds, and $q_{ij} = 0$ for fully failed bonds. The final nodal damage is

$$D_i = 1 - \frac{A_i}{B_i}. \quad (74)$$

Because the family table stores directed bonds, the code also checks the reciprocal entry (j, i) . If either fail_{ij} or the reciprocal entry has failed, the bond is treated as failed and carries no force.

If $\text{fail}_{ij} = 0$ when the bond loop reaches the bond, the bond is already fully failed and the active code uses

$$\Delta p_{ij} = 0, \quad (75)$$

$$W_{ij} = 0, \quad (76)$$

$$\Delta G_{ij}^{\text{fracture}} = 0, \quad (77)$$

$$q_{ij} = 0. \quad (78)$$

That is, already failed bonds continue to count in the denominator B_i , but they do not add force, strain energy, or damage numerator.

13.2 Common Elastic Contribution

Whenever a branch decides that a bond is elastic, the equations entering the code are

$$\Delta p_{ij} = c s_{ij} w_{ij} n_{ij}, \quad (79)$$

$$W_{ij} = \frac{1}{4} c s_{ij}^2 r_{ij}^0 w_{ij}, \quad (80)$$

$$\Delta G_{ij}^{\text{fracture}} = 0, \quad (81)$$

$$q_{ij} = 1. \quad (82)$$

13.3 Common Softening Contribution

For the linear, bilinear, Cornelissen, and exponential softening branches, the irreversibility variable is the history stretch

$$\kappa_{ij}(t) = \max_{\tau \leq t} s_{ij}^+(\tau), \quad (83)$$

stored in the code as `bond_s_history[i][j]`. The prescribed envelope is written as

$$f_{\text{env},ij}(\kappa) = c p(\kappa). \quad (84)$$

For the implemented softening laws, the code has $p(\kappa) = s_0 \mu_{ij}$ after the elastic limit. The degraded unloading/reloading stiffness is

$$g(\kappa) = \frac{p(\kappa)}{\kappa}, \quad k_{\text{unload},ij} = c g(\kappa) = \frac{f_{\text{env},ij}}{\kappa}. \quad (85)$$

If the bond is on the envelope,

$$\Delta p_{ij} = f_{\text{env},ij} w_{ij} n_{ij}. \quad (86)$$

If the bond is unloading or reloading,

$$\Delta p_{ij} = k_{\text{unload},ij} s_{ij} w_{ij} n_{ij}. \quad (87)$$

In both cases, the strain-energy contribution stored by the code is the directed-bond version of the free energy:

$$W_{ij} = \frac{1}{4} c g(\kappa_{ij}) s_{ij}^2 r_{ij}^0 w_{ij}. \quad (88)$$

The factor 1/4 appears because the implementation accumulates directed bond contributions at nodes; it is the pair-split counterpart of the usual $\frac{1}{2} c r g(\kappa) s^2$ bond free energy.

The damage numerator uses

$$q_{ij} = \mu_{ij}. \quad (89)$$

13.4 Common Complete-Failure Contribution

For the brittle, linear, bilinear, Cornelissen, and fallback branches, once the code decides that a bond has completely failed, it sets

$$\text{fail}_{ij} = 0, \quad (90)$$

and uses

$$\Delta p_{ij} = 0, \quad (91)$$

$$W_{ij} = 0, \quad (92)$$

$$q_{ij} = 0. \quad (93)$$

For the brittle, bilinear, Cornelissen, exponential, and fallback branches, the current code adds the branch-local complete-failure value stored in `d_fracture_energy`. For `degradation_model = 1`, the code instead uses the incremental history-energy update described in the linear-softening subsection. For linear softening at complete failure, the limiting accumulated value is

$$G_{ij}^{\text{fracture}} = \frac{1}{4} c s_0 s_c r_{ij}^0 w_{ij}. \quad (94)$$

13.5 Critical Stretch

The initial critical stretch is

$$s_0 = \sqrt{\frac{3G_c}{E\delta}}. \quad (95)$$

The current fracture parameter is

$$G_c = 10^{-8}, \quad (96)$$

and the softening energy parameter is

$$R_d = G_c. \quad (97)$$

The complete-failure stretch s_c depends on the selected model.

For linear softening,

$$s_c = 1.5 \left(s_0 + \frac{2R_d}{cs_0} \right). \quad (98)$$

For bilinear softening,

$$s_c = s_0 + \frac{2R_d}{cs_0}. \quad (99)$$

For the Cornelissen law,

$$s_c = s_0 + \frac{5.1361R_d}{cs_0}. \quad (100)$$

For the exponential law,

$$s_c = 10s_0 \quad (101)$$

is used as a numerical reference cutoff in `main.cpp`; the active exponential branch now uses this value as the complete-failure cutoff.

The bilinear transition stretch is

$$s_k = s_0 + 0.15(s_c - s_0), \quad (102)$$

with residual ratio

$$\beta = 0.25. \quad (103)$$

13.6 Elastic Model

For `degradation_model` < 0, the bond remains elastic:

$$f_{ij} = c s_{ij} n_{ij} w_{ij}. \quad (104)$$

13.7 Brittle Model

For `degradation_model` = 0, the bond is elastic while

$$s_{ij} \leq s_0. \quad (105)$$

If

$$s_{ij} > s_0, \quad (106)$$

then

$$f_{ij} = 0, \quad \text{fail}_{ij} = 0. \quad (107)$$

The fracture-energy accumulator receives

$$G_{ij}^{\text{fracture}} = \frac{1}{4} c s_0^2 r_{ij}^0 w_{ij}. \quad (108)$$

13.8 Linear Softening Model

If

$$s_{ij}^{\max} \leq s_0 \quad (109)$$

the bond is elastic.

If

$$s_0 < s_{ij}^{\max} < s_c, \quad (110)$$

the degradation factor is

$$\mu_{ij} = \frac{s_c - s_{ij}^{\max}}{s_c - s_0}. \quad (111)$$

The envelope force magnitude is

$$f_{\text{env}} = cs_0\mu_{ij}. \quad (112)$$

On the loading envelope,

$$f_{ij} = f_{\text{env}}n_{ij}w_{ij}. \quad (113)$$

For unloading or reloading below the historical maximum, the secant stiffness is

$$k_{\text{unload}} = \frac{f_{\text{env}}}{s_{ij}^{\text{max}}}, \quad (114)$$

and

$$f_{ij} = k_{\text{unload}}s_{ij}n_{ij}w_{ij}. \quad (115)$$

If

$$s_{ij}^{\text{max}} \geq s_c, \quad (116)$$

the bond is fully failed:

$$f_{ij} = 0, \quad \text{fail}_{ij} = 0. \quad (117)$$

For `degradation_model` = 1, fracture energy is stored incrementally with a per-bond history value `bond_fracture_energy_history[i][j]`. During partial softening the code computes

$$W_{\text{input}} = \frac{1}{2}cs_0^2 + \frac{1}{2}cs_0(1 + \mu_{ij})(s_{ij}^{\text{max}} - s_0), \quad (118)$$

$$W_{\text{recoverable}} = \frac{1}{2}f_h s_{ij}^{\text{max}}, \quad f_h = cs_0\mu_{ij}, \quad (119)$$

and

$$G_{\text{now}} = \frac{1}{2}(W_{\text{input}} - W_{\text{recoverable}})r_{ij}^0w_{ij}. \quad (120)$$

Only the positive increment

$$\Delta G = G_{\text{now}} - G_{\text{history}} \quad (121)$$

is added to `fracture_energy[i]`, and then G_{history} is updated. At complete failure, the total target value is

$$G_{\text{total}} = \frac{1}{4}cs_0s_cr_{ij}^0w_{ij}. \quad (122)$$

13.9 Bilinear Softening Model

For $s_0 < s_{ij}^{\text{max}} < s_k$,

$$\mu_{ij} = 1 - (1 - \beta) \frac{s_{ij}^{\text{max}} - s_0}{s_k - s_0}. \quad (123)$$

For $s_k \leq s_{ij}^{\text{max}} < s_c$,

$$\mu_{ij} = \beta \frac{s_c - s_{ij}^{\text{max}}}{s_c - s_k}. \quad (124)$$

Then the same envelope and unloading/reloading formulas are used.

13.10 Cornelissen Softening Model

Define

$$\eta = \frac{s_{ij}^{\max} - s_0}{s_c - s_0}. \quad (125)$$

The degradation factor is

$$\mu_{ij} = (1 + (3\eta)^3) e^{-6.93\eta} - 28\eta e^{-6.93}. \quad (126)$$

The code clamps this value:

$$\mu_{ij} = \max(0, \mu_{ij}). \quad (127)$$

13.11 Exponential Softening Model

The exponential law uses

$$\alpha = \frac{cs_0}{R_d}, \quad (128)$$

and

$$\mu_{ij} = \exp[-\alpha (s_{ij}^{\max} - s_0)]. \quad (129)$$

If

$$s_{ij}^{\max} \geq s_c, \quad (130)$$

the bond is fully failed:

$$f_{ij} = 0, \quad \text{fail}_{ij} = 0. \quad (131)$$

14 Nodal Damage Variable

The nodal damage-like variable is

$$D_i = 1 - \frac{\sum_{j \in \mathcal{H}_i} \mu_{ij} V \text{fac}_{ij}}{\sum_{j \in \mathcal{H}_i} V \text{fac}_{ij}}. \quad (132)$$

For intact elastic bonds, $\mu_{ij} = 1$. For softened bonds, the current history-based μ_{ij} is used. Fully failed bonds contribute to the denominator but not the numerator.

Thus $D_i = 0$ means all counted bonds around node i are intact; larger values indicate local softening or bond failure. Grip nodes are assigned $D_i = 0$ because their force loop is skipped.

15 Stress Measure

The stress-like output is a peridynamic force-moment measure:

$$\sigma_i = C_\sigma \sum_{j \in \mathcal{H}_i} f_{ij} \xi_{ij}. \quad (133)$$

The active coefficient is

$$C_\sigma = \frac{1}{2}. \quad (134)$$

The optional virtual substrate force is included in equilibrium only when the source flag

$$\text{enable_substrate_coupling} \quad (135)$$

is true. It is not included in this bar-bar stress moment.

16 Output Strain

The saved strain is a finite-difference post-processing field, not the peridynamic bond stretch.

At the left endpoint,

$$\varepsilon_0 = \frac{u_1 - u_0}{\Delta x}. \quad (136)$$

For interior nodes,

$$\varepsilon_i = \frac{1}{2} \left[\frac{u_i - u_{i-1}}{\Delta x} + \frac{u_{i+1} - u_i}{\Delta x} \right]. \quad (137)$$

At the right endpoint,

$$\varepsilon_{N-1} = \frac{u_{N-1} - u_{N-2}}{\Delta x}. \quad (138)$$

17 Energy Quantities

For an elastic intact bond, the strain-energy contribution at node i is

$$W_{ij} = \frac{1}{4} c s_{ij}^2 r_{ij}^0 w_{ij}. \quad (139)$$

For softened unloading or reloading, the code uses

$$W_{ij} = \frac{1}{4} k_{\text{unload}} s_{ij}^2 r_{ij}^0 w_{ij}. \quad (140)$$

For `degradation_model = 1`, fracture energy is accumulated incrementally from the bond history. The code stores a previous per-bond value `bond_fracture_energy_history[i][j]`, computes the current history value G_{now} , and adds only

$$\Delta G = G_{\text{now}} - G_{\text{history}} \quad (141)$$

when this increment is positive. This prevents the same history energy from being added repeatedly during ADR iterations. In the current code, the other damage branches keep their branch-local complete-failure increments.

If substrate coupling is enabled, the virtual substrate energy is

$$W_i^{\text{sub}} = \sum_j \frac{1}{4} c_{\text{int}} \left(s_{ij}^{\text{sub}} \right)^2 r_{ij}^{0,\text{sub}} V. \quad (142)$$

For the current default tensile-bar scenario, this term is identically zero.

The kinetic energy is

$$K_i = \frac{1}{2} m_i v_i^2. \quad (143)$$

The current lumped mass-like value is

$$m_i = 1. \quad (144)$$

Damping dissipation is accumulated as

$$\mathcal{D}_i^{n+1} = \mathcal{D}_i^n + m_i c_d v_i^2 \Delta t. \quad (145)$$

The total stored nodal energy density is

$$E_i^{\text{tot}} = W_i + W_i^{\text{sub}} + K_i + \mathcal{D}_i + G_i^{\text{fracture}}. \quad (146)$$

18 Dynamic Relaxation Solver

Each outer load increment is equilibrated by an explicit damped dynamic relaxation loop. The active equation is

$$m_i \ddot{u}_i + c_d \dot{u}_i = p_i + b_i. \quad (147)$$

Since $b_i = 0$,

$$m_i \ddot{u}_i + c_d \dot{u}_i = p_i. \quad (148)$$

The acceleration used in the code is

$$a_i^n = \frac{p_i^n + b_i - c_d v_i^n}{m_i}. \quad (149)$$

The active update is

$$v_i^{n+1} = v_i^n + a_i^n \Delta t, \quad (150)$$

$$u_i^{n+1} = u_i^n + v_i^{n+1} \Delta t. \quad (151)$$

The mechanical time step is

$$\Delta t = \text{dt_mechanical_bar}. \quad (152)$$

It is computed from

$$\tau = \frac{L}{\sqrt{E/\rho}}, \quad (153)$$

and

$$\text{dt_mechanical_bar} = \frac{\text{dt_mechanical}}{\tau}. \quad (154)$$

With the current defaults $L = E = \rho = 1$,

$$\tau = 1, \quad \text{dt_mechanical_bar} = 5 \times 10^{-4}. \quad (155)$$

The code also computes `dt_pd`, `dt_lambda`, and `dt_bar`, but the active mechanical update uses `dt_mechanical_bar`.

19 Damping and Convergence

The default damping coefficient is

$$c_d = 5. \quad (156)$$

This is the active path because

$$\text{fixed_damping_coefficient} = \text{true}. \quad (157)$$

If adaptive damping is enabled, the code estimates

$$c_{\text{opt}} = 2 \sqrt{\frac{\text{num}}{\text{den}}}, \quad (158)$$

caps the result, and smooths it as

$$c_d \leftarrow 0.7c_d + 0.3c_{\text{opt}}. \quad (159)$$

The relaxation loop stops when

$$\max_i |v_i| \leq 10^{-6}. \quad (160)$$

The maximum iteration count is

$$100000. \quad (161)$$

20 Algorithm Summary

The initialization sequence is:

1. Read constants from `main.cpp`, `variable_initialization.h`, and `time_settings.h`.
2. Allocate arrays with `matrices`.
3. Build the one-dimensional grid and family list with `build_Geometry`.
4. Compute surface correction factors with `surface_correction_factors`.
5. Precompute bond geometry and weights with `precompute_bond_invariants`.
6. Initialize displacement and velocity fields to zero.
7. Set the lumped mass-like coefficient to 1 for every node.
8. Call `mechanical`.

For each outer load increment:

1. Increase the applied strain by $\Delta\varepsilon$.
2. Add the affine predictor $0.75\Delta\varepsilon x_i$.
3. Reset velocities and old force state for dynamic relaxation.
4. Run the inner relaxation loop until convergence.
5. Write final fields for the increment.
6. Optionally detect first crack and stop five increments later.

For each dynamic relaxation iteration:

1. Reimpose displacement and zero velocity on the first and last three nodes.
2. Compute bar-bar bond stretches and forces.
3. Update historical maximum stretch for each bond.
4. Apply the selected degradation law.
5. Add virtual substrate/interface force only if substrate coupling is enabled.
6. Compute nodal damage, stress, strain energy, and substrate energy.
7. Update damping if adaptive damping is enabled.
8. Update velocity and displacement explicitly.
9. Compute kinetic energy, damping dissipation, total energy, and convergence speed.
10. Compute finite-difference output strain.
11. Write optional diagnostics or snapshot buffers.

21 Output Files

21.1 Configuration Files

For every outer increment, the code writes

$$\text{result/confK.dat.} \quad (162)$$

Each row contains

$$x_i, \quad u_i, \quad \varepsilon_i, \quad D_i. \quad (163)$$

21.2 OVITO Files

For every increment, the code writes

$$\text{ovito_files/for_ovito_K.xyz.} \quad (164)$$

The columns are

$$x, y, u_x, u_y, x_{\text{def}}, y_{\text{def}}, D, \sigma, W, \varepsilon. \quad (165)$$

Because the active model is one-dimensional, y , u_y , and y_{def} are written as zero.

21.3 Final ADR Field Matrices

The final converged fields for each increment are written to:

- Final_ADR_stress.txt
- Final_ADR_strain.txt
- Final_ADR_strain_energy.txt
- Final_ADR_kinetic_energy.txt
- Final_ADR_fracture_energy.txt
- Final_ADR_damping_dissipation_energy.txt
- Final_ADR_substrate_energy.txt

Rows correspond to outer increments; columns correspond to nodes.

21.4 Energy Histories

System-level energy histories during the ADR iterations are written to:

- total_energy_density_matrix.txt
- total_kinetic_energy_density_matrix.txt
- total_damping_dissipation_energy_density_matrix.txt
- total_fracture_energy_density_matrix.txt
- total_strain_energy_density_matrix.txt
- total_substrate_energy_density_matrix.txt

Each row is one outer increment; each column is one ADR iteration.

21.5 Stress Convergence

When `track_stress_convergence` is true, the code writes

$$\text{convergence_of_stress.txt.} \quad (166)$$

It tracks σ at the node closest to

$$x = 0.3. \quad (167)$$

21.6 Snapshots and Diagnostics

The snapshot mechanism can write all ADR-step fields for selected increments:

- `stressK.txt`
- `strainK.txt`
- `strain_energyK.txt`
- `kinetic_energyK.txt`
- `fracture_energyK.txt`
- `damping_dissipation_energyK.txt`
- `substrate_energyK.txt`
- `total_energy_densityK.txt`

By default, `write_snaps` is set to a value not reached by ordinary runs.

If `track_numerical_noise` is enabled, diagnostic stress, force, displacement, and maximum-stretch traces are also written for selected nodes.

22 First-Crack Early Stop

The early-stop mode is disabled by default:

$$\text{stop_after_first_crack} = \text{false.} \quad (168)$$

When enabled, first crack is detected when any node satisfies

$$D_i > 0. \quad (169)$$

The simulation then stops five increments after that first detection.

23 Defect Handling

`DefectUtils.h` can mark nodes in a circular defect region:

$$(x_i - x_c)^2 + (y_i - y_c)^2 \leq r_{\text{defect}}^2. \quad (170)$$

It computes a reduced critical stretch

$$s_0^{\text{defect}} = s_0 \times \text{defect_strength_factor.} \quad (171)$$

The default defect values are:

Quantity	Variable	Value
Enabled	<code>introduce_defect</code>	<code>false</code>
Center	<code>defect_x_center</code> , <code>defect_y_center</code>	<code>(0,0)</code>
Radius	<code>defect_radius</code>	Δx
Strength factor	<code>defect_strength_factor</code>	0.6

Important implementation detail: the force loop currently comments out the code that would replace s_0 with s_0^{defect} . Therefore enabling the defect mask alone does not change the active bond failure or softening law.

24 Post-Processing

The Python script `plot_strain.py` reads `result/conf*.dat`. Example uses:

```
python3 plot_strain.py --steps 1 20 80
python3 plot_strain.py --steps 1 20 80 --displacement --damage
python3 plot_strain.py --movie peridynamics.mp4 --damage --strain-zoom
```

The script expects the columns

$$x, \quad u, \quad \varepsilon, \quad D. \quad (172)$$

If an older file has only three columns, it fills damage with zeros.

25 Active Versus Dormant Code

Dormant item	Current status
Two-dimensional film/substrate grid	Commented out; the active model uses $y = 0$ for all nodes.
Thermal solve	Thermal constants exist, but no active thermal loop is called.
Explicit traction boundary conditions	Present as comments; active loading is fixed-left and pulled-right displacement control.
Separate deformable substrate points	Disabled by default; optional virtual affine substrate coupling remains in the solver.
<code>write_ovito_files.h</code>	Not included by active <code>main.cpp</code> .
Defect critical stretch	The mask exists, but the local critical-stretch substitution is commented out.
<code>dt_bar</code> stability estimate	Computed for reference, but active ADR uses <code>dt_mechanical_bar</code> .
Half-step ADR variant	Older half-step velocity update code is commented out.
heaviside helper	Defined in <code>main.cpp</code> , but unused.

26 Important Implementation Notes

1. The active model is mostly nondimensional and uses variables with the suffix `_bar`.
2. The pair (i, j) and the pair (j, i) have separate entries in `fail` and `bond_s_history`.

3. The stress output includes only bar-bar bond force moments, not the optional virtual substrate force.
4. Grip nodes are skipped by the force loop; damage can occur in the free bar bonds once the selected law reaches its damage threshold.
5. Grip nodes are excluded from the active force loop and report zero stress and damage from that path.
6. `maxfam` must be large enough for every family. With $m = 3$ in one dimension, `maxfam=20` is comfortably sufficient.
7. Generated files in `result/`, `ovito_files/`, and the energy text files are run products rather than source files.